Practical 2: Data Preprocessing Techniques

Practical Information

Attribute	Details
Practical Number	2
Title	Data Preprocessing Techniques
Duration	2 Hours
Course Outcomes (COs)	CO1, CO2
Description	Learn and implement various data preprocessing and cleaning techniques.

Key Topics Covered

- Missing values
- Outlier detection
- Feature scaling
- Encoding
- Data transformation

Instructions for Students

1. **Complete all tasks** outlined in this practical
2. **Write your code** in the cells provided below
3. **Add comments** to explain your code
4. **Include proper output** (prints, visualizations, etc.)
5. **Submit your completed notebook** before the deadline

Your Work Starts Here

Use the cell(s) below to write your implementation:

```
# Practical 2: Data Preprocessing Techniques

import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder

data = {
    'Age': [22, 25, None, 30, 28],
    'Salary': [25000, 30000, 28000, None, 32000],
```

```
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Female']
}

df = pd.DataFrame(data)
print("Original Dataset:\n", df)

# Step 2: Handle Missing Values (Mean Imputation)
df['Age'].fillna(df['Age'].mean(), inplace=True)
df['Salary'].fillna(df['Salary'].mean(), inplace=True)

# Step 3: Encode Categorical Data
le = LabelEncoder()
df['Gender'] = le.fit_transform(df['Gender'])

# Step 4: Feature Scaling
scaler = StandardScaler()
df[['Age', 'Salary']] = scaler.fit_transform(df[['Age', 'Salary']])

print("\nPreprocessed Dataset:\n", df)
```

```
Original Dataset:
     Age   Salary  Gender
0  22.0  25000.0    Male
1  25.0  30000.0  Female
2   NaN  28000.0  Female
3  30.0      NaN    Male
4  28.0  32000.0  Female

Preprocessed Dataset:
         Age    Salary  Gender
0 -1.567636 -1.621266       1
1 -0.461069  0.540422       0
2  0.000000 -0.324253       0
3  1.383208  0.000000       1
4  0.645497  1.405098       0
C:\Users\SHRAVYA\AppData\Local\Temp\ipykernel_6432\656862164.py:20: FutureWarnin
The behavior will change in pandas 3.0. This inplace method will never work beca

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met


  df['Age'].fillna(df['Age'].mean(), inplace=True)
C:\Users\SHRAVYA\AppData\Local\Temp\ipykernel_6432\656862164.py:21: FutureWarnin
The behavior will change in pandas 3.0. This inplace method will never work beca

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met


  df['Salary'].fillna(df['Salary'].mean(), inplace=True)
```

```
# Handling Missing Values

import pandas as pd
import numpy as np

data = {
    'Marks': [85, 90, np.nan, 75, np.nan],
    'Attendance': [90, np.nan, 85, 88, 92]
}

df = pd.DataFrame(data)
print("Before Handling Missing Values:\n", df)

# Method 1: Mean Imputation
df['Marks'].fillna(df['Marks'].mean(), inplace=True)
df['Attendance'].fillna(df['Attendance'].mean(), inplace=True)

print("\nAfter Handling Missing Values:\n", df)
```

```
Before Handling Missing Values:
    Marks  Attendance
0   85.0        90.0
1   90.0         NaN
2    NaN        85.0
3   75.0        88.0
4    NaN        92.0

After Handling Missing Values:
        Marks  Attendance
0  85.000000       90.00
1  90.000000       88.75
2  83.333333       85.00
3  75.000000       88.00
4  83.333333       92.00
C:\Users\SHRAVYA\AppData\Local\Temp\ipykernel_6432\2735675351.py:18: FutureWarni
The behavior will change in pandas 3.0. This inplace method will never work beca

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met


  df['Marks'].fillna(df['Marks'].mean(), inplace=True)
C:\Users\SHRAVYA\AppData\Local\Temp\ipykernel_6432\2735675351.py:19: FutureWarni
The behavior will change in pandas 3.0. This inplace method will never work beca

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.met


  df['Attendance'].fillna(df['Attendance'].mean(), inplace=True)
```

```
# Program to remove outliers using IQR method
```

```
import pandas as pd

data = {
    'Marks': [35, 40, 42, 45, 48, 50, 95, 100]
}

df = pd.DataFrame(data)
print("Original Dataset:\n", df)

# Step 2: Calculate Q1, Q3 and IQR
Q1 = df['Marks'].quantile(0.25)
Q3 = df['Marks'].quantile(0.75)
IQR = Q3 - Q1

# Step 3: Define lower and upper limits
lower_limit = Q1 - 1.5 * IQR
upper_limit = Q3 + 1.5 * IQR

# Step 4: Remove outliers
df_cleaned = df[(df['Marks'] >= lower_limit) & (df['Marks'] <= upper_limit)]

print("\nQ1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("\nLower Limit:", lower_limit)
print("Upper Limit:", upper_limit)

print("\nDataset after removing outliers:\n", df_cleaned)
```

```
Original Dataset:
    Marks
0     35
1     40
2     42
3     45
4     48
5     50
6     95
7    100

Q1: 41.5
Q3: 61.25
IQR: 19.75

Lower Limit: 11.875
Upper Limit: 90.875

Dataset after removing outliers:
    Marks
0     35
1     40
```

```
2     42
3     45
4     48
5     50
```